

교과목 2 : 데이터 분석과 머신러닝 / 딥러닝

단원 3 : 딥러닝

DAY6. 딥러닝 모델 저장 및 활용

목 차

1. 딥러닝 모델 저장 및 활용		3
2. 오차 역전파법		13
3. 컴퓨터비전 모델 활용		18

1. 딥러닝 모델 저장 및 활용

딥러닝 모델을 학습시키면 컴퓨터는 데이터의 패턴을 학습하여 **가중치(Weight)** 와 **편향(Bias)** 이라는 값을 만들어 낸다. 이 값들이 바로 모델이 학습한 결과이다.

하지만 학습이 끝난 후 컴퓨터를 종료하면 메모리에 있던 학습 결과는 사라진다. 따라서 학습한 내용을 파일로 저장해 두어야 한다. 이것을 **모델 저장(Model Saving)** 이라고 한다.

예를 들어 고양이와 강아지를 구분하는 딥러닝 모델을 5시간 동안 학습했다고 가정하자. 학습이 끝난 후 모델을 저장하지 않으면 다음 날 다시 5시간 동안 처음부터 학습해야 한다. 반면 모델을 저장해 두면 저장된 파일을 불러오기만 하면 바로 사용할 수 있다.

왜 모델을 저장하는가?

모델을 저장하는 가장 큰 이유는 학습 시간을 절약하기 위해서이다.

예를 들어 다음과 같은 상황을 생각해 볼 수 있다.

1. 딥러닝 모델 학습
2. 모델 저장
3. 컴퓨터 종료
4. 다음 날 모델 불러오기
5. 즉시 예측 수행

만약 저장하지 않았다면 다음 날 다시 처음부터 학습해야 한다.

따라서 실제 인공지능 서비스에서는 반드시 모델을 저장한다.

모델 저장 후 할 수 있는 일

저장된 모델은 여러 가지 목적으로 활용할 수 있다.

1). 예측 수행

학습된 모델을 이용하여 새로운 데이터를 분류하거나 값을 예측할 수 있다.

예를 들어

- 손글씨 숫자 인식
- 고양이/강아지 분류
- 주택 가격 예측
- 주가 예측

등을 수행할 수 있다.

2). 추가 학습

기존 모델을 불러온 뒤 새로운 데이터로 다시 학습할 수 있다.

이를 **재학습(Fine-Tuning)** 이라고 한다.

예를 들어

- 기존 데이터 1만 개 학습
- 새로운 데이터 1천 개 추가

가 발생했을 때 처음부터 학습하는 대신 기존 모델을 불러와서 이어서 학습한다.

3). 웹 서비스 배포

학습한 모델을 웹 서버에 올려 실제 사용자가 사용할 수 있게 한다.

예를 들어

- ChatGPT
- 얼굴 인식 시스템
- 음성 인식 시스템
- 번역 시스템

모두 저장된 모델을 서버에서 불러와 사용한다.

모델 저장 과정

딥러닝 모델 저장 과정은 다음과 같다.

데이터 준비

↓

모델 생성

↓

모델 학습

↓

모델 저장

↓

프로그램 종료

↓

모델 불러오기

↓

예측 수행

PyTorch에서 모델 저장

PyTorch에서는 일반적으로 학습된 가중치만 저장한다.

모델

- └─ 가중치(weight)
- └─ 편향(bias)
- └─ 구조(코드)

PyTorch는 보통

- 모델 구조는 코드에 보관
- 학습 결과는 파일에 저장

하는 방식을 사용한다.

예를 들어

```
torch.save(model.state_dict(), "model.pth")
```

를 실행하면 model.pth 파일이 생성된다.

이 파일 안에는 학습된 가중치 값이 저장된다.

PyTorch에서 모델 불러오기

모델을 사용할 때는

1. 모델 구조 생성
2. 저장된 가중치 불러오기
3. 예측 수행

순서로 진행한다.

```
model.load_state_dict(torch.load("model.pth"))
```

이 코드는 저장된 학습 결과를 모델에 넣는 과정이다.

TensorFlow에서 모델 저장

TensorFlow는 모델 전체를 저장할 수 있다.

```
model.save("model.keras")
```

실행 후 model.keras 파일이 생성된다.

이 파일 안에는

- 모델 구조
- 가중치
- 손실 함수 정보
- 최적화 알고리즘 정보

등이 함께 저장된다.

TensorFlow에서 모델 불러오기

저장된 모델은 다음과 같이 불러온다.

```
loaded_model = tf.keras.models.load_model("model.keras")
```

그러면 학습 당시의 모델이 그대로 복원된다.

PyTorch와 TensorFlow 비교

구분	PyTorch	TensorFlow
저장 방식	가중치 위주 저장	모델 전체 저장
저장 파일	.pth	.keras
불러오기	모델 구조 필요	바로 사용 가능
유연성	높음	사용 편리
실무 활용	연구, 개발	서비스 배포

실제 현업에서는 무엇을 저장할까?

실무에서는 단순히 모델만 저장하지 않는다.

다음 정보도 함께 저장한다.

모델 파일

model.pth

또는

model.keras

데이터 전처리 정보

예를 들어

이미지 크기 = 224×224

픽셀값 = 0~1 정규화

같은 정보

클래스 정보

예를 들어

0 = 고양이

1 = 강아지

2 = 토끼

같은 분류 정보

학습 설정 정보

예를 들어

학습률 = 0.001

Epoch = 100

Batch Size = 32

등의 정보

쉽게 비유하면

딥러닝 모델은 학생과 비슷하다.

- 학습 전 → 아무것도 모르는 학생
- 학습 중 → 공부하는 학생
- 학습 완료 → 시험을 잘 보는 학생
- 모델 저장 → 공부한 내용을 책으로 기록
- 모델 불러오기 → 책을 다시 꺼내 공부 내용 활용

즉, 모델 저장은 딥러닝이 학습한 지식을 파일로 보관하는 과정이고, 모델 불러오기는 그 지식을 다시 사용하는 과정이라고 이해하면 된다.

정리하면, 딥러닝 모델 저장은 학습이 끝난 모델의 **구조, 가중치, 학습 설정, 전처리 방식** 등을 파일로 보관하여 나중에 다시 사용하는 과정이다. 모델을 저장해 두면 매번 처음부터 다시 학습하지 않아도 되고, 저장된 모델을 불러와서 예측, 추가 학습, 서비스 배포에 활용할 수 있다.

PyTorch에서는 일반적으로 모델 전체를 저장하기보다 `state_dict` 방식으로 **학습된 가중치만 저장**하는 방법을 권장한다. 이 방식은 모델 구조 코드는 따로 유지하고, 저장 파일에는 가중치와 편향 값만 담는다. 모델을 불러올 때는 같은 구조의 클래스를 먼저 만든 뒤 `load_state_dict()`로 가중치를 넣어야 한다. PyTorch 공식 문서도 모델 가중치를 불러올 때 같은 모델 인스턴스를 먼저 생성해야 한다고 설명한다. ([PyTorch Docs](#))

TensorFlow/Keras에서는 `model.save()`를 사용하면 모델 구조, 가중치, 컴파일 정보 등을 하나의 파일 또는 폴더 형태로 저장할 수 있다. TensorFlow 공식 문서에서는 학습 도중이나 학습 후 모델을 저장하면 긴 학습 시간을 반복하지 않고 이어서 활용할 수 있다고 설명한다. ([TensorFlow](#))

1. PyTorch 모델 저장 및 활용 예제

```
# PyTorch 딥러닝 모델 저장 및 불러오기 예제

import torch
import torch.nn as nn
import torch.optim as optim

# 입력 데이터와 정답 데이터 생성
# X는 입력값, y는 정답값이다.
X = torch.tensor([[1.0], [2.0], [3.0], [4.0]])
y = torch.tensor([[2.0], [4.0], [6.0], [8.0]])

# 간단한 선형 회귀 모델 클래스 정의
class SimpleModel(nn.Module):
    def __init__(self):
        super().__init__()

        # 입력 1개를 받아 출력 1개를 만드는 선형 계층
        self.linear = nn.Linear(1, 1)
```



```

def forward(self, x):
    # 입력 데이터를 선형 계층에 통과시켜 예측값 생성
    return self.linear(x)

# 모델 객체 생성
model = SimpleModel()

# 평균제곱오차 손실함수 사용
criterion = nn.MSELoss()

# SGD 최적화 알고리즘 사용
optimizer = optim.SGD(model.parameters(), lr=0.01)

# 모델 학습
for epoch in range(1000):
    # 모델 예측
    pred = model(X)

    # 예측값과 정답값의 차이를 손실로 계산
    loss = criterion(pred, y)

    # 이전 기울기 초기화
    optimizer.zero_grad()

    # 역전파로 기울기 계산
    loss.backward()

    # 가중치와 편향 업데이트
    optimizer.step()

# 학습된 모델의 가중치 저장
torch.save(model.state_dict(), "pytorch_model.pth")

```

```
print("PyTorch 모델 저장 완료")
```

```
# 저장된 PyTorch 모델 불러오기 및 예측
```

```
import torch
```

```
import torch.nn as nn
```

```
# 저장할 때 사용했던 모델 구조와 동일하게 정의
```

```
class SimpleModel(nn.Module):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.linear = nn.Linear(1, 1)
```

```
    def forward(self, x):
```

```
        return self.linear(x)
```

```
# 빈 모델 객체 생성
```

```
loaded_model = SimpleModel()
```

```
# 저장된 가중치 불러오기
```

```
loaded_model.load_state_dict(torch.load("pytorch_model.pth", weights_only=True))
```

```
# 예측 모드로 변경
```

```
# Dropout, BatchNorm 등이 있을 때 학습 모드와 예측 모드를 구분하기 위해 필요하다.
```

```
loaded_model.eval()
```

```
# 새로운 입력 데이터
```

```
new_data = torch.tensor([[5.0]])
```

```
# 기울기 계산 없이 예측 수행
```

```
with torch.no_grad():
```

```
    result = loaded_model(new_data)
```

```
print("예측 결과:", result.item())
```

2. TensorFlow 모델 저장 및 활용 예제

```
# TensorFlow 딥러닝 모델 저장 예제

import tensorflow as tf
import numpy as np

# 입력 데이터와 정답 데이터 생성
X = np.array([[1.0], [2.0], [3.0], [4.0]])
y = np.array([2.0, 4.0, 6.0, 8.0])

# 간단한 Sequential 모델 생성
model = tf.keras.Sequential([
    # 입력값 1개를 받아 출력값 1개를 만드는 Dense 계층
    tf.keras.layers.Dense(1, input_shape=(1,))
])

# 모델 컴파일
# optimizer는 가중치 업데이트 방법, loss는 오차 계산 방법이다.
model.compile(
    optimizer=tf.keras.optimizers.SGD(learning_rate=0.01),
    loss="mse"
)

# 모델 학습
model.fit(X, y, epochs=1000, verbose=0)

# 모델 전체 저장
model.save("tensorflow_model.keras")
```

```
print("TensorFlow 모델 저장 완료")
```

```
# 저장된 TensorFlow 모델 불러오기 및 예측
```

```
import tensorflow as tf
```

```
import numpy as np
```

```
# 저장된 모델 불러오기
```

```
loaded_model = tf.keras.models.load_model("tensorflow_model.keras")
```

```
# 새로운 입력 데이터
```

```
new_data = np.array([[5.0]])
```

```
# 예측 수행
```

```
result = loaded_model.predict(new_data)
```

```
print("예측 결과:", result[0][0])
```

정리하면 PyTorch는 보통 **모델 구조 코드 + 가중치 파일**을 함께 관리하는 방식이고, TensorFlow/Keras는 **모델 전체를 하나의 .keras 파일로 저장**하여 비교적 간단하게 다시 불러오는 방식이다. 실무에서는 모델 파일뿐 아니라 입력 데이터 전처리 방법, 클래스 이름, 학습에 사용한 하이퍼파라미터도 함께 저장해야 같은 결과를 안정적으로 재현할 수 있다.

2. 오차 역전파법

오차 역전파법(Backpropagation)은 딥러닝 신경망이 학습할 때 사용하는 핵심 알고리즘이다. 모델이 예측한 값과 실제 정답 사이의 오차를 계산한 후, 그 오차가 발생한 원인을 찾아 각 가중치(Weight)를 얼마나 수정해야 하는지를 계산하는 방법이다.

쉽게 말하면 학생이 시험을 보고 틀린 문제를 확인한 뒤, 왜 틀렸는지 분석하여 다음 시험에서는 같은 실수를 하지 않도록 공부 방법을 수정하는 과정과 비슷하다.

오차 역전파법이 필요한 이유

딥러닝 모델은 처음 학습을 시작할 때 가중치가 무작위 값으로 설정된다.

예를 들어 고양이 사진을 입력했는데 모델이 강아지라고 예측했다고 가정해 보자.

입력 : 고양이 사진

실제 정답 : 고양이

모델 예측 : 강아지

모델은 예측이 틀렸으므로 오차(Error)가 발생한다.

이때 모델은 어떤 가중치가 문제였을까?

어느 부분을 얼마나 수정해야 할까?

를 계산해야 한다.

이 계산을 수행하는 방법이 바로 오차 역전파법이다.

신경망 학습 과정

딥러닝 학습은 일반적으로 다음 순서로 진행된다.

1. 입력 데이터 입력

↓

2. 순전파(Forward Propagation)

↓

3. 예측 결과 생성

↓

4. 손실함수 계산

↓

5. 오차 역전파 수행

↓

6. 가중치 수정

↓

7. 반복 학습

순전파(Forward Propagation)

순전파는 입력 데이터가 입력층에서 출력층 방향으로 이동하는 과정이다.

예를 들어

입력층 → 은닉층 → 출력층

구조가 있다고 하자.

입력값 : $x = 2$

가중치 : $w = 3$

라고 하면 출력은 : $y = x \times w$ 이다.

따라서 $y = 2 \times 3 = 6$ 이 된다.

손실함수(Loss Function)

모델이 예측한 값과 실제 정답의 차이를 계산한다.

예를 들어

실제값 = 10

예측값 = 6

이라면 오차가 존재한다.

평균제곱오차(MSE)는 계산하면

$Loss = (10 - 6)^2 = 16$ 이다.

손실값이 클수록 예측이 많이 틀렸다는 의미이다.

역전파(Backpropagation)

손실값을 계산한 후에는 손실이 왜 발생했는가? 를 분석해야 한다.

출력층에서 계산된 오차를 거꾸로 전달하면서 각 가중치가 오차에 얼마나 영향을 주었는지를 계산한다.

그래서 이름도 역(Backward) + 전파(Propagation) 이다.

미분이 필요한 이유

가중치를 수정하려면

가중치를 증가시켜야 하나?

감소시켜야 하나?

얼마나 수정해야 하나?

를 알아야 한다.

이를 위해 손실함수를 가중치에 대해 미분한다.

예를 들어

$$Loss = (10 - 2w)^2 \quad \text{이라면}$$

가중치 (w)에 대한 기울기는

$$dLoss/dw \quad \text{가 된다.}$$

기울기는 손실이 증가하는 방향을 의미한다.

경사하강법과의 관계

역전파는 기울기를 계산한다.

경사하강법(Gradient Descent)은 계산된 기울기를 이용하여 가중치를 수정한다.

가중치 업데이트 식은

$$w_{new} = w_{old} - \eta \frac{\partial Loss}{\partial w}$$

이다.

여기서

- w : 가중치
- η : 학습률(Learning Rate)
- $\frac{\partial Loss}{\partial w}$: 기울기

이다.

즉, 역전파 → 기울기 계산

경사하강법 → 가중치 수정 역할을 한다.

연쇄법칙(Chain Rule)

딥러닝 신경망은 여러 층으로 구성된다.

입력층

↓

은닉층 1

↓

은닉층 2

↓

출력층

오차를 이전 층으로 전달하려면 미분 결과를 계속 연결해야 한다.

이때 사용하는 것이 연쇄법칙이다.

예를 들어

$$z = xw$$

$$a = \text{sigmoid}(z)$$

$$\text{Loss} = (y-a)^2$$

이라면

$$d\text{Loss}/dw = d\text{Loss}/da \times da/dz \times dz/dw$$

처럼 연결하여 계산한다.

이를 연쇄법칙이라고 한다.

오차 역전파의 실제 의미

출력층에서 오차 = 0.8 이 발생했다고 가정하자.

은닉층의 뉴런 A 가 오차의 60%에 영향을 주었다면

$$0.8 \times 0.6 = 0.48$$

만큼의 책임이 있다고 판단한다.

은닉층의 뉴런 B 가 오차의 20%에 영향을 주었다면

$$0.8 \times 0.2 = 0.16$$

만큼의 책임이 있다고 판단한다.

이러한 방식으로 각 뉴런과 가중치의 책임 정도를 계산하여 수정한다.

Epoch 가 증가할수록 일어나는 일

학습 초기

손실값 = 5.2

이었다면

역전파와 경사하강법을 반복하면서

손실값 = 3.8

손실값 = 2.1

손실값 = 0.9

손실값 = 0.2

처럼 점점 감소한다.

이는 모델이 정답에 가까워지고 있다는 의미이다.

오차 역전파의 장점

1. 빠른 학습

가중치를 효율적으로 수정할 수 있다.

2. 대규모 신경망 학습 가능

수백만 개 이상의 파라미터도 학습할 수 있다.

3. 자동 미분 지원

PyTorch 와 TensorFlow 가 자동으로 계산해 준다.

```
loss.backward()
```

한 줄로 역전파가 수행된다.

오차 역전파법은 출력층에서 계산된 오차를 입력층 방향으로 거꾸로 전달하면서 각 가중치가 오차에 얼마나 영향을 주었는지 계산하고, 그 결과를 이용하여 가중치를 수정하는 딥러닝 학습의 핵심 알고리즘이다.

3. 컴퓨터비전 모델 활용

컴퓨터 비전이란?



컴퓨터 비전은 컴퓨터가 디지털 이미지 또는 비디오에서 높은 수준의 이해를 위한 결과를 얻을 수 있는 방법을 다루는 과학 분야입니다. 공학적 관점에서 볼 때, 인간의 시각 시스템이 할 수 있는 작업을 이해하고 자동화하려고 연구하는 분야입니다.

컴퓨터 비전 작업에는 디지털 이미지를 획득, 처리, 분석 및 이해하는 방법, 수치 또는 기호 정보(예: 의사 결정 형태)를 생성하기 위해 현실 세계에서 고차원 데이터를 추출하는 방법이 포함됩니다. 이러한 맥락에서 이해한다는 것은 시각 이미지(망막의 입력)를 사고 과정에 이치에 맞고 적절한 행동을 유도할 수 있는 세계에 대한 묘사로 변형시키는 것을 의미한다. 이러한 이미지 이해는 기하학, 물리학, 통계 및 학습 이론의 도움으로 구성된 모델을 사용하여 이미지 데이터에서 기호 정보를 분리하는 것으로 볼 수 있습니다.

컴퓨터 비전의 과학적 분야는 이미지에서 정보를 추출하는 인공지능 시스템 이론과 관련이 있습니다. 이미지 데이터는 비디오 시퀀스, 여러 카메라의 뷰, 3D 스캐너의 다차원 데이터 또는 의료 스캐닝 장치와 같은 다양한 형태를 취할 수 있습니다. 컴퓨터 비전의 기술 분야는 컴퓨터 비전 시스템 구축에 이론과 모델을 적용하려고 합니다.

컴퓨터 비전의 하위 영역에는 장면 재구성, 객체 감지, 이벤트 감지, 비디오 추적, 객체 인식, 3D 포즈 추정, 학습, 인덱싱, 동작 추정, 시각적 서빙, 3D 장면 모델링 및 이미지 복원이 포함됩니다.

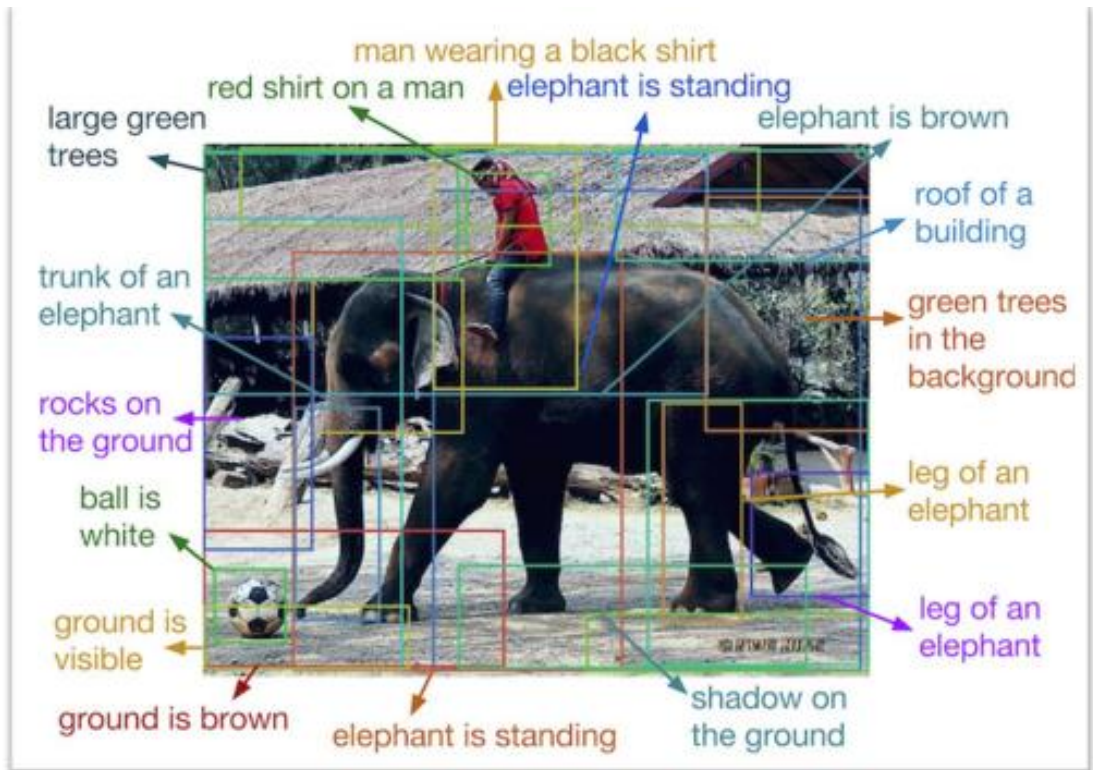
컴퓨터 비전(computer vision)은 디지털 이미지와 동영상 속 사물을 식별하는 기술입니다. 생명체는 시각 피질로 이미지를 처리하기 때문에, 많은 연구자들이 포유류의 시각 피질 구조를 모델로 삼아 이미지 인식을 수행할 신경망을 설계했습니다. 이러한 생물학적 연구는 1950년대로 거슬러 올라갑니다.

지난 20년간 컴퓨터 비전 기술은 놀랍게 발전했습니다. 아직 완벽하지는 않지만 컴퓨터 비전 시스템 중에는 99% 정확성을 보이는 것도 있고, 모바일 장치에서 제법 잘 실행되는 것도 있습니다.

시각 구현을 위한 신경망 분야의 돌파구는 1998년 얀 르쿤(Yann LeCun)의 르넷-5(LeNet-5)였습니다. 르넷-5는 7단계 컨볼루션 신경망(convolutional neural network, 이하 CNN)으로서 32x32픽셀 이미지로 디지털화된 손글씨 숫자를 인식할 수 있었습니다. 이보다 더 높은 해상도의 이미지를 분석하려면 르넷-5 네트워크가 더 많은 신경세포와 계층으로 확장해야 했습니다.

오늘날 가장 뛰어난 이미지 분류 모델의 경우, 다양한 사물을 컬러 HD 해상도로 식별 가능합니다. 순수 심층 신경망(deep neural networks, DNN)뿐만 아니라 혼합형 시각 모델도 가끔 사용됩니다. 혼합형 모델은 특정 하위 작업을 수행하는 고전적인 머신러닝 알고리즘에 딥 러닝을 결합한 것입니다.

시각 구현에 있어서 기본적인 이미지 분류 이외의 다른 문제들(예: 국소화로 이미지 분류, 사물 탐지, 사물 구획화, 이미지 형식 전송, 이미지 컬러화, 이미지 재구성, 이미지 초해상, 이미지 합성)은 딥 러닝으로 해결되었습니다.



DenseCap "고밀도 캡션"소프트웨어의 출력, 코끼리를 타는 남자의 사진을 분석

이 이미지는 **컴퓨터 비전(Computer Vision)**이 사진을 어떻게 이해하는지 보여주는 예시입니다. 사람은 사진을 한 번 보면 "코끼리 위에 사람이 올라타 있다"라고 쉽게 이해하지만, 컴퓨터는 처음부터 그렇게 이해하지 못합니다. 컴퓨터는 사진 속의 작은 영역들을 하나씩 분석하여 무엇이 있는지 찾아낸 후 전체 장면을 이해합니다.

1. 원본 사진의 내용

사진에는 다음과 같은 요소들이 포함되어 있습니다.

- 코끼리 1 마리
- 코끼리 위에 올라탄 사람 1 명
- 축구공 1 개
- 나무
- 건물 지붕
- 땅
- 그림자

사람은 이것들을 한 번에 인식하지만, 컴퓨터는 각각을 따로 찾아야 합니다.

2. 색깔 사각형의 의미

사진 위에 보이는 여러 개의 색깔 사각형은 **객체 탐지(Object Detection)** 결과입니다.

예를 들어,

- 파란색 상자 → 코끼리
- 초록색 상자 → 사람
- 갈색 상자 → 땅
- 하늘색 상자 → 건물 지붕

처럼 특정 영역이 무엇인지 표시합니다.

즉,

이 영역 = 사람

이 영역 = 코끼리

이 영역 = 축구공

과 같이 컴퓨터가 판단한 결과를 보여주는 것입니다.

3. 문장의 의미

이미지 주변에 적혀 있는 문장들은 컴퓨터가 자동으로 생성한 설명입니다.

예를 들면

elephant is standing

→ 코끼리가 서 있다.

man wearing a black shirt

→ 검은색 셔츠를 입은 남자

elephant is brown

→ 코끼리는 갈색이다.

roof of a building

→ 건물의 지붕

shadow on the ground

→ 땅 위의 그림자

를 의미합니다.

즉, 컴퓨터가 사진을 보고 사람처럼 설명문을 만든 것입니다.

4. Dense Captioning 기술

일반적인 이미지 분류는

사진 = 코끼리

정도만 알려줍니다.

객체 탐지는

코끼리 위치

사람 위치

공 위치

를 알려줍니다.

하지만 이 그림은 그보다 더 발전한

Dense Captioning(고밀도 캡션 생성) 기술입니다.

Dense Captioning 은

위치 찾기 + 무엇인지 인식 + 문장으로 설명

을 동시에 수행합니다.

예를 들어

이 영역 = 코끼리 다리

이 영역 = 코끼리 몸통

이 영역 = 남자의 빨간 셔츠

이 영역 = 공

와 같이 매우 세밀하게 분석합니다.

5. 컴퓨터 비전 처리 과정

이 이미지가 분석되는 과정을 순서대로 설명하면 다음과 같습니다.

① 이미지 입력

카메라

↓

사진 획득

② 객체 탐지

CNN 이나 YOLO 같은 딥러닝 모델이

사람

코끼리

공

나무

를 찾습니다.

③ 특징 추출

객체의 특징을 분석합니다.

예)

코끼리 색상

코끼리 크기

사람 옷 색상

공 모양

④ 객체 인식

분석 결과를 기반으로

이것은 사람

이것은 코끼리

이것은 축구공

이라고 판단합니다.

⑤ 캡션 생성

자연어 생성 모델이

A man is riding an elephant. (남자가 코끼리를 타고 있다.)

와 같은 문장을 생성합니다.

6. 딥러닝 모델

현재는 대부분 CNN 과 Transformer 기반 모델이 사용됩니다.

대표적인 모델은

- CNN
- Faster R-CNN
- YOLO
- SSD
- Mask R-CNN
- Vision Transformer(ViT)
- BLIP
- GPT-4o 계열 멀티모달 모델

등이 있습니다.

이러한 모델들은 수백만 장의 이미지로 학습하면서

사람은 어떤 모양인가?

코끼리는 어떤 특징을 가지는가?

공은 어떤 형태인가?

를 학습합니다.

7. 이미지가 보여주는 핵심 개념

이 그림은 컴퓨터 비전의 주요 기술을 한 번에 보여주는 대표적인 예시입니다.

기술	그림에서 수행하는 작업
이미지 분류(Image Classification)	사진에 코끼리가 있음
객체 검출(Object Detection)	코끼리, 사람, 공의 위치 찾기
객체 인식(Object Recognition)	무엇인지 판별
이미지 캡셔닝(Image Captioning)	설명 문장 생성
장면 이해(Scene Understanding)	전체 상황 파악
Dense Captioning	여러 영역을 세밀하게 설명

컴퓨터 비전 연구 분야

□ 영상의 화질 개선



Filtering App



HDR

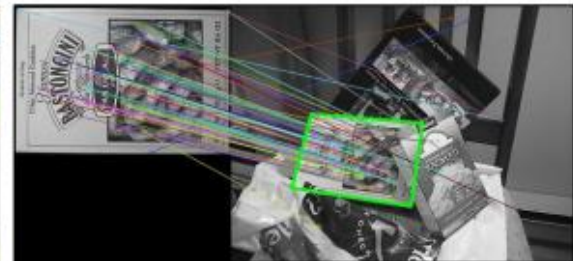


Image Noise Remove

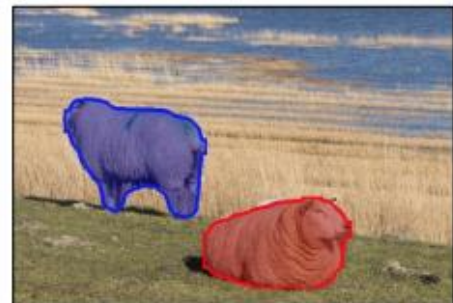


Super Resolution

□ 객체 검출(Object detection)

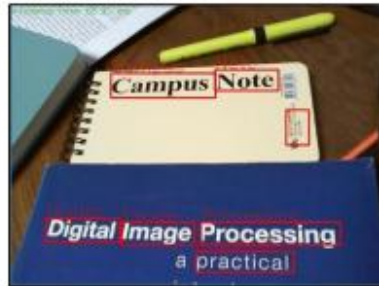


□ 분할(Segmentation)



컴퓨터 비전 연구 분야

□ 인식(Recognition)



□ 움직임 정보 추출 및 추적



분야	목적	대표 기술
영상 화질 개선	영상 품질 향상	Filtering, HDR, Super Resolution
객체 검출	물체 위치 찾기	YOLO, Faster R-CNN
분할	객체 경계 추출	U-Net, Mask R-CNN
인식	객체 종류 판별	CNN, ResNet, ViT
추적	움직임 분석	Tracking, Pose Estimation

현재 자율주행 자동차, 의료 영상 분석, 얼굴 인식, 스마트 공장, CCTV 분석, 로봇 비전 등의 시스템은 이러한 기술들을 함께 사용하여 주변 환경을 이해하고 판단합니다

컴퓨터 비전 응용 분야

□ 머신 비전(machine vision)

- ✓ 공장 자동화: 제품의 불량 검사, 위치 확인, 측정 등
- ✓ 높은 정확도와 빠른 처리 시간 요구
- ✓ 조명, 렌즈, 필터, 실시간 (Real-time) 처리

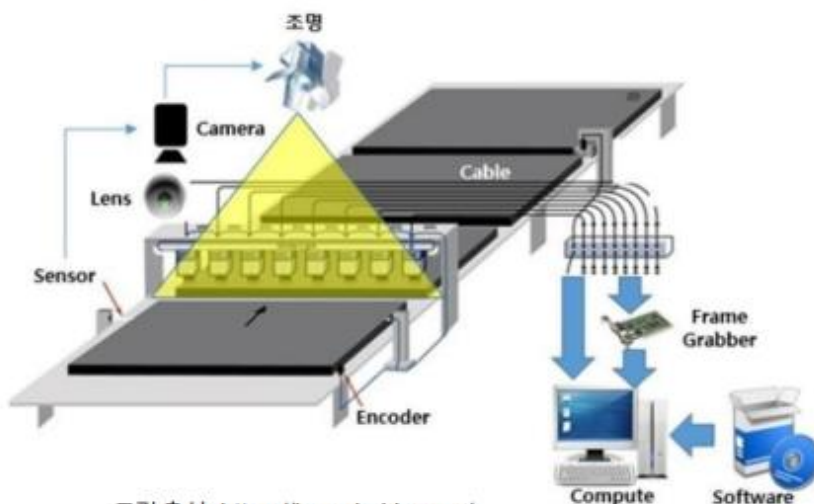


그림 출처: <https://laonple.blog.me/>

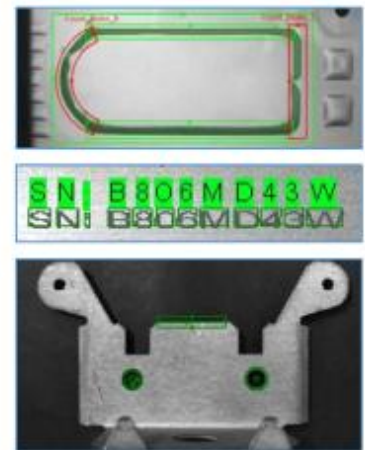


그림 출처: <http://www.cognex.com>

□ 인공지능(AI) 서비스

- ✓ 인공지능 로봇과 자율 주행 자동차
- ✓ 입력 영상을 객체와 배경으로 분할 ⑦ 객체와 배경 인식 ⑦ 상황 인식 ⑦ 로봇과 자동차의 행동 지시
- ✓ Computer Vision + Sensor Fusion + Deep Learning
- ✓ Amazon Go / 구글, 테슬라의 자율 주행 자동차



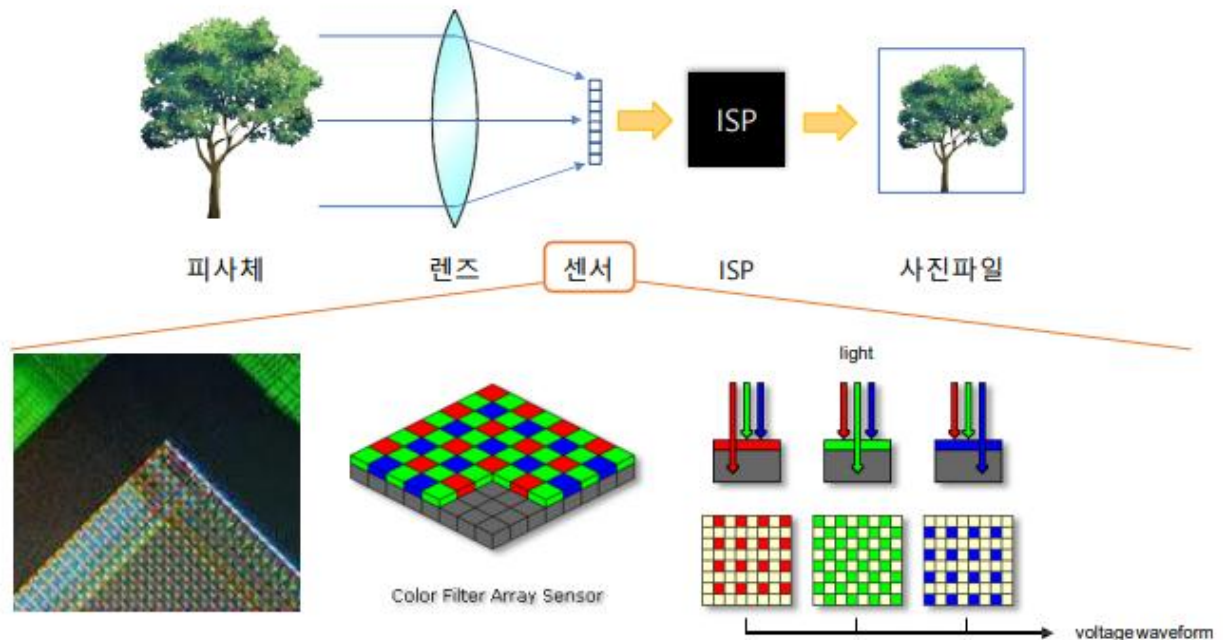
<https://www.youtube.com/watch?v=NrmMk1Myrxc>



<https://www.youtube.com/watch?v=wuhbqcMzOaw>

영상의 획득

□ 디지털 카메라에서 영상의 획득 과정



디지털 카메라는 피사체에서 반사된 빛을 렌즈로 모아 이미지 센서에서 전기 신호로 변환하고, ISP(Image Signal Processor)가 색상 복원·노이즈 제거·화질 개선을 수행한 후 최종적으로 JPG와 같은 사진 파일로 저장한다.

컴퓨터 비전은 사진을 분석하기 전에 먼저 사진이 어떻게 만들어지는지 이해해야 합니다.

왜냐하면

- 노이즈 발생 원인
- 색상 왜곡 원인
- 밝기 변화 원인
- 해상도 한계

를 알아야 영상 처리 알고리즘을 설계할 수 있기 때문입니다.

예를 들어

노이즈 제거

초해상도

객체 검출

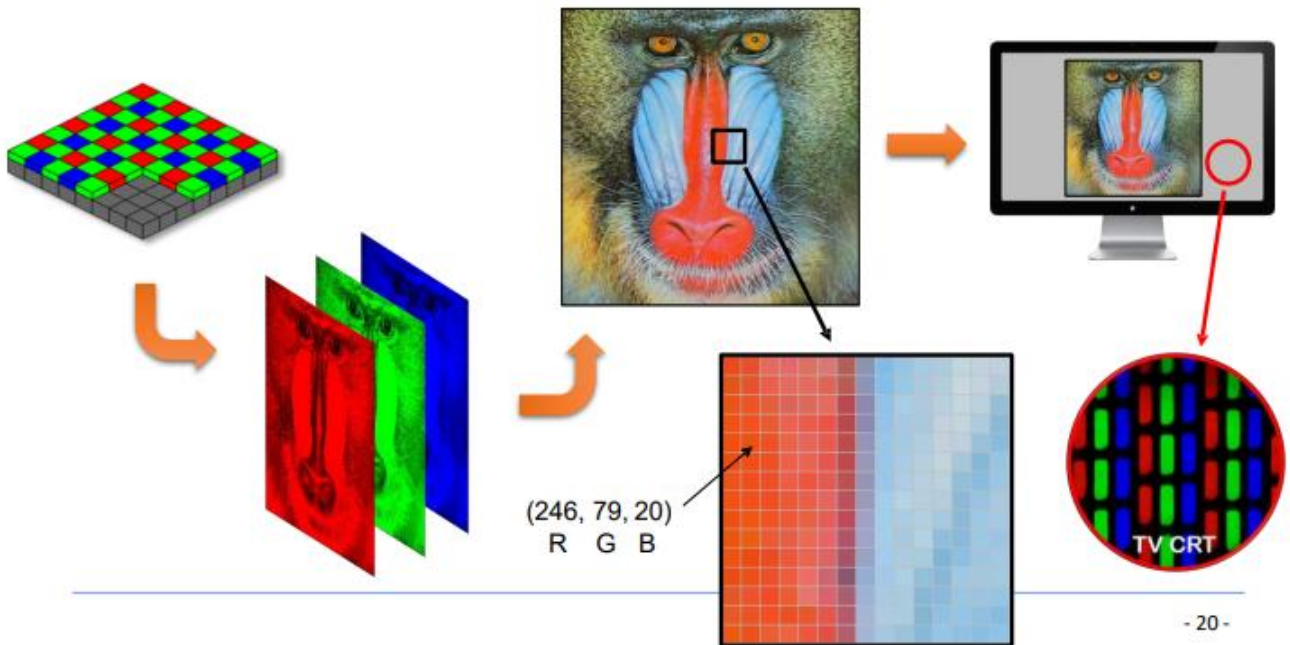
얼굴 인식

기술들은 모두 이 영상 획득 과정을 기반으로 발전한 기술들입니다.

영상의 표현 방법

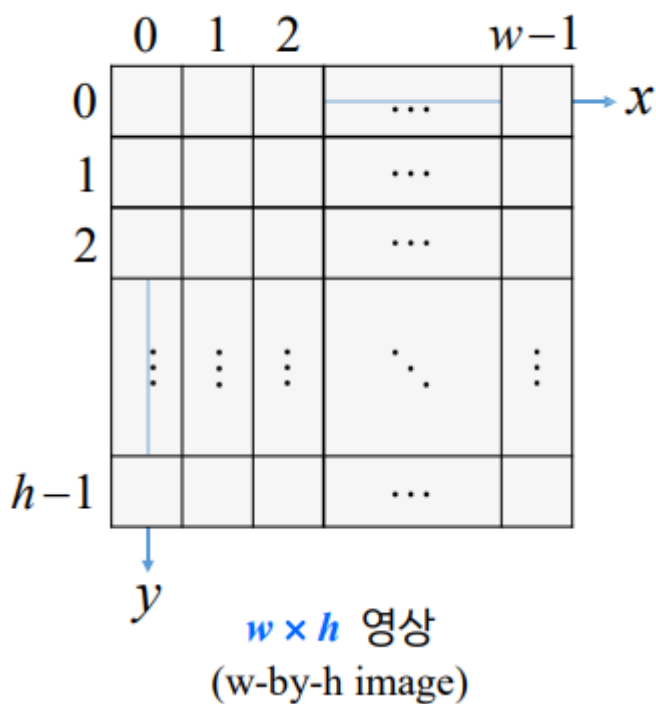
□ 영상(image)이란?

✓ 픽셀(pixel)이 바둑판 모양의 격자에 나열되어 있는 형태 (2차원 행렬)



- 20 -

□ 영상에서 사용되는 좌표계



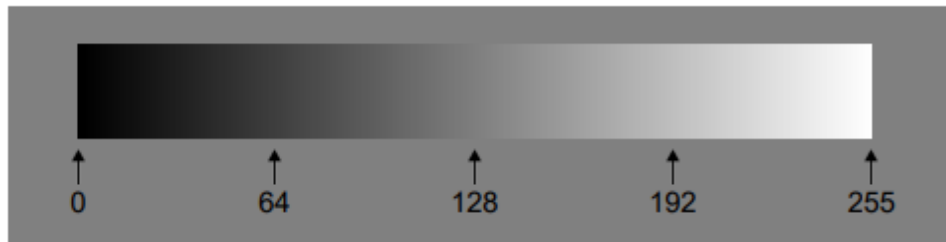
$$\mathbf{A} = \begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,N} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,N} \\ \vdots & \vdots & \ddots & \vdots \\ a_{M,1} & a_{M,2} & \cdots & a_{M,N} \end{bmatrix}$$

$M \times N$ 행렬
(m-by-n matrix)

그레이스케일 영상과 컬러 영상

□ 그레이스케일 값의 범위(Grayscale level)

- ✓ 그레이스케일 영상에서 하나의 픽셀은 0부터 255 사이의 정수 값을 가짐.
 - 0 : 가장 어두운 밝기(검정색)
 - 255 : 가장 밝은 밝기(흰색)



- ✓ C/C++ 에서 unsigned char 로 표현 (1 byte)

```
typedef unsigned char BYTE;           // Windows
typedef unsigned char uint8_t;        // Linux
typedef unsigned char uchar;          // OpenCV
```

- ✓ Python에서는? → `numpy.uint8`

□ 그레이스케일 영상에서 픽셀 값 분포의 예

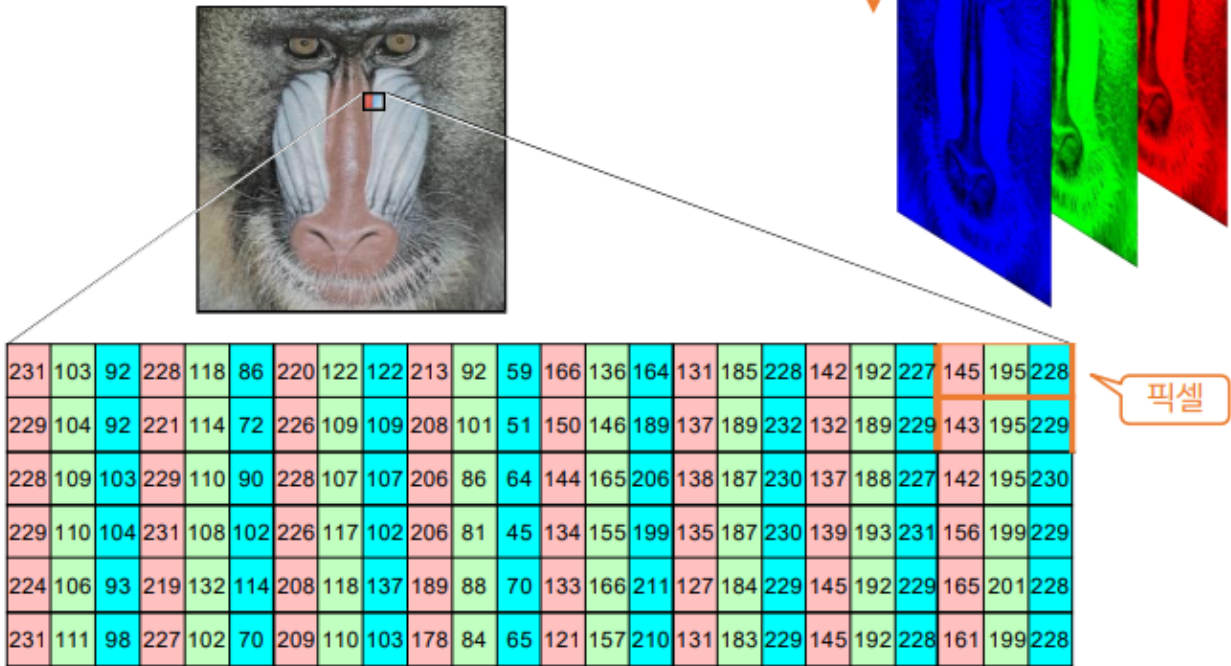


187	187	187	194	197	173	77	25	19	19
190	187	190	191	158	37	15	14	20	20
187	182	180	127	32	16	13	16	14	12
184	186	172	100	20	13	15	18	13	18
186	190	187	127	18	14	15	14	12	10
189	192	192	148	16	15	11	10	10	9
192	195	181	37	13	10	10	10	10	10
189	194	54	14	11	10	10	10	9	8
189	194	19	16	11	11	10	10	9	9
192	88	12	11	11	10	10	10	9	9

픽셀

그레이스케일 영상과 컬러 영상

□ 트루컬러 영상에서 픽셀 값 분포의 예

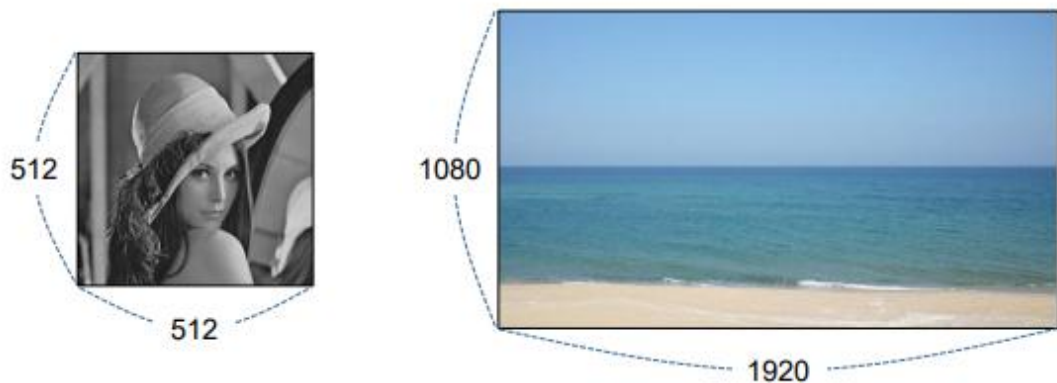


컬러 영상은 수많은 픽셀로 구성되며, 각 픽셀은 Red(R), Green(G), Blue(B)의 세 가지 색상 값(0~255)을 저장하고, 컴퓨터는 이러한 RGB 숫자들의 집합으로 이미지를 표현하고 처리한다.

영상의 표현 방법

□ 영상 데이터 크기 분석

- ✓ 그레이스케일 영상: (가로 크기) × (세로 크기) Bytes
- ✓ 트루컬러 영상: (가로 크기) × (세로 크기) × 3 Bytes



$$512 \times 512 = 262144 \text{ Bytes}$$

$$1920 \times 1080 \times 3 = 6220800 \text{ Bytes} \\ \approx 6 \text{ MBytes}$$

영상 파일 형식 특징

BMP

- 픽셀 데이터를 압축하지 않고 그대로 저장, 파일 용량이 큰 편
- 파일 구조가 단순해서 별도의 라이브러리 도움 없이 직접 파일 입출력 프로그래밍 가능

JPG

- 주로 사진과 같은 컬러 영상을 저장하기 위해 사용
- 손실 압축(lossy compression)
- 압축률이 좋아서 파일 용량이 크게 감소, 디지털 카메라 사진 포맷으로 주로 사용됨

GIF

- 256 색상 이하의 영상을 저장
- 일반 사진을 저장 시 화질 열화가 심함
- 무손실 압축(lossless compression)
- 움직이는 GIF 지원

PNG

- Portable Network Graphics
- 무손실 압축 (트루컬러 영상도 무손실 압축)
- 알파 채널(투명도)을 지원

OpenCV 개요



□ What is OpenCV?

- ✓ Open source
- ✓ Computer vision & machine learning
- ✓ Software library

□ Why OpenCV?

- ✓ BSD license ...Free for academic & commercial use
- ✓ Multiple interface ...C, C++, Python, Java, JavaScript, MATLAB, etc.
- ✓ Multiple platform ...Windows, Linux, Mac OS, iOS, Android
- ✓ Optimized ...CPU instructions, Multi-core processing, OpenCL, CUDA
- ✓ Popular ...More than 15 million downloads
- ✓ Usage ... Stitching streetview images, detecting intrusions, monitoring mine equipment, helping robots navigate and pick up objects, Interactive art, etc.

OpenCV(Open Source Computer Vision Library) 는 이미지 처리와 컴퓨터 비전을 위한 대표적인 오픈소스 라이브러리입니다.

주요 특징은 다음과 같습니다.

- 무료로 사용할 수 있는 오픈소스 소프트웨어
- 컴퓨터 비전 및 머신러닝 기능 제공
- Python, C++, Java 등 다양한 언어 지원
- Windows, Linux, Mac, Android, iOS 등 다양한 운영체제 지원
- CPU, 멀티코어, CUDA 등을 활용한 고속 처리 지원

주요 기능은 다음과 같습니다.

- 이미지 읽기 및 저장
- 영상 처리 및 필터링
- 객체 검출(Object Detection)
- 얼굴 인식(Face Recognition)
- 문자 인식(OCR)

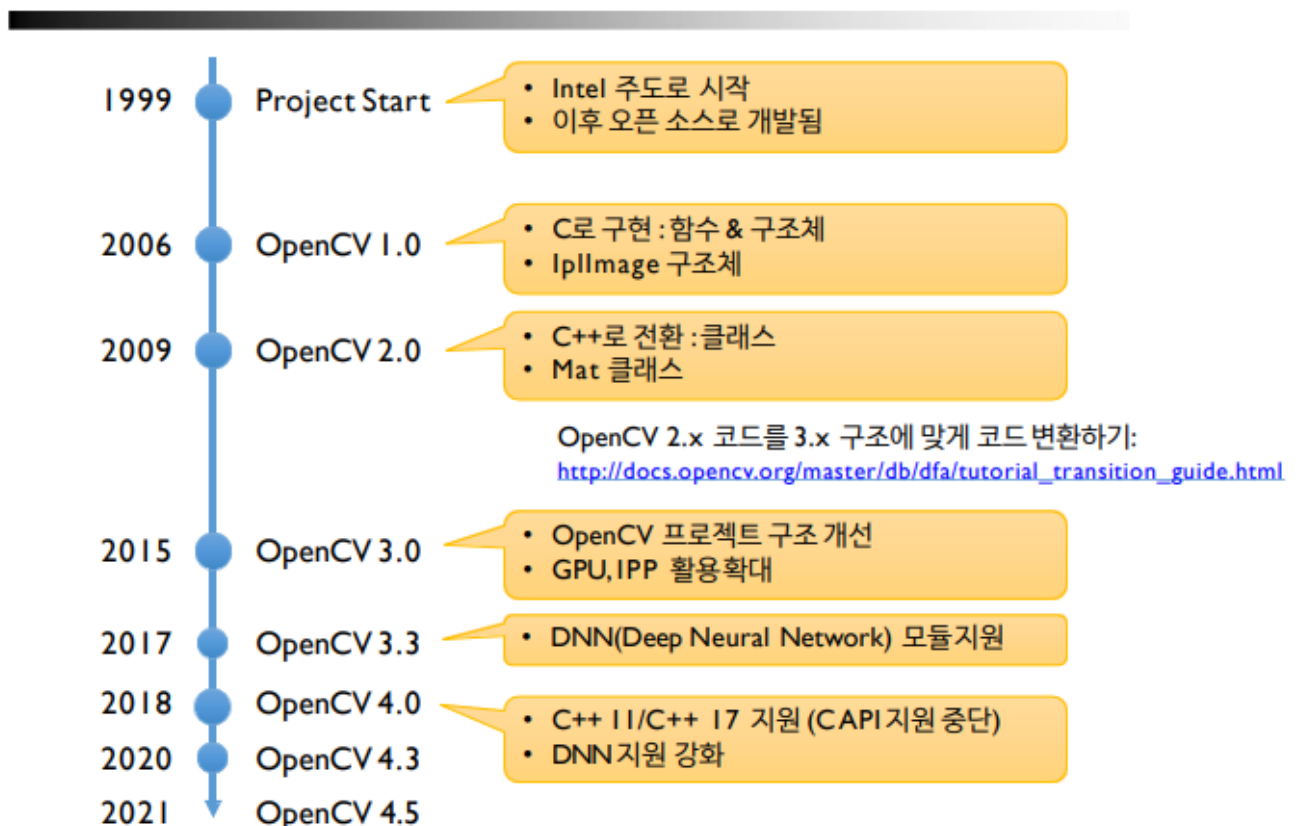
- 객체 추적(Object Tracking)
- 딥러닝 모델 실행

활용 분야는 다음과 같습니다.

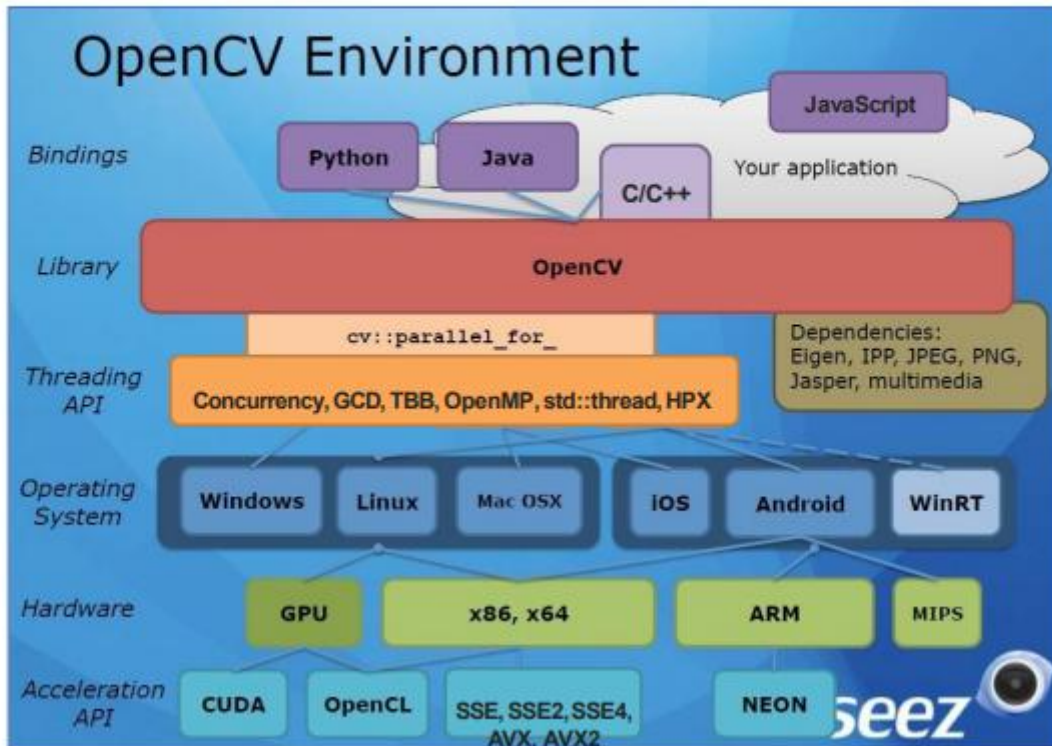
- 자율주행 자동차
- CCTV 영상 분석
- 얼굴 인식 시스템
- 산업용 머신 비전
- 로봇 비전
- 의료 영상 분석
- 증강현실(AR)

OpenCV는 이미지와 영상을 분석하고 처리하기 위한 무료 오픈소스 컴퓨터 비전 라이브러리로, 객체 검출, 얼굴 인식, 영상 처리 등 다양한 기능을 제공하는 대표적인 개발 도구이다.

OpenCV 역사



OpenCV 구성



OpenCV 는 Python, C++, Java 등의 언어에서 사용할 수 있는 컴퓨터 비전 라이브러리로, 다양한 운영체제와 하드웨어를 지원하며 CPU·GPU 가속 기술을 활용해 빠른 이미지 및 영상 처리를 수행할 수 있도록 구성되어 있다.

OpenCV 모듈

OpenCV는 모듈식 구조를 가지고 있습니다. 즉, 패키지에는 여러 공유 또는 정적 라이브러리가 포함됩니다. 다음 모듈을 사용할 수 있습니다.

- **핵심 기능 (core)** - 조밀한 다차원 배열 Mat 및 다른 모든 모듈에서 사용되는 기본 기능을 포함하여 기본 데이터 구조를 정의하는 소형 모듈입니다.
- **이미지 처리 (imgproc)** - 선형 및 비선형 이미지 필터링, 기하학적 이미지 변환(크기 조정, 아핀 및 원근 변환, 일반 테이블 기반 재매핑), 색 공간 변환, 히스토그램 등을 포함하는 이미지 처리 모듈입니다.
- **비디오 분석 (비디오)** - 모션 추정, 배경 빼기 및 객체 추적 알고리즘을 포함하는 비디오 분석 모듈입니다.
- **카메라 보정 및 3D 재구성 (calib3d)** - 기본 다중 뷰 기하학 알고리즘, 단일 및 스테레오 카메라 보정, 물체 포즈 추정, 스테레오 대응 알고리즘 및 3D 재구성 요소.
- **2D 기능 프레임워크 (features2d)** - 두드러진 기능 감지기, 설명자 및 설명자 일치자.
- **객체 감지 (objdetect)** - 미리 정의된 클래스(예: 얼굴, 눈, 머그, 사람, 자동차 등)의 객체 및 인스턴스 감지.
- **고급 GUI (highgui)** - 간단한 UI 기능에 대한 사용하기 쉬운 인터페이스입니다.
- **비디오 I/O (videoio)** - 비디오 캡처 및 비디오 코덱에 대한 사용하기 쉬운 인터페이스입니다.
- ... FLANN 및 Google 테스트 래퍼, Python 바인딩 등과 같은 기타 도우미 모듈.

OpenCV-Python 설치

□ pip 명령으로 OpenCV-Python 설치하기

✓ 설치 명령

```
> pip install opencv-python
```

- `<PYTHON_PATH>\Lib\site-packages` 아래에 `cv2`, `opencv_python` - `4.5.5.64.dist-info` 폴더가 생성되고, 그 아래에 `pyd` 파일이 저장됨. (`cv2.pyd`)
- 설치되는 OpenCV 버전은 <https://pypi.org/> 에서 확인 가능
- `numpy` 패키지도 함께 설치됨
- OpenCV 추가 모듈도 함께 사용하려면 `opencv-contrib-python` 패키지를 설치

- OpenCV 4.2.0 버전에서 `cv2.imread()` 함수를 사용하여 컬러 영상을 그레이스케일 형식으로 불러올 때 픽셀 값이 잘못 설정되는 문제가 있음
- `cv2.imread()` 함수가 정상 동작하는 OpenCV를 설치하려면 아래 명령 사용

```
> pip install opencv-python==4.1.0.25
```

- OpenCV 프로그래밍 기초
 - ✓ 영상 불러와서 출력하기
 - ✓ 영상 데이터 다루기 (생성, 복사, 크롭, 합성)
 - ✓ 컬러 영상 처리: 색 공간 변환
 - ✓ 기하학적 변환 (크기 변환, 회전 변환)
 - ✓ OpenCV 그리기 함수
- 카메라 & 동영상 처리
 - ✓ 카메라 & 동영상 프레임 처리
 - ✓ 동영상 파일 저장하기

OpenCV API

□ 영상 파일 불러오기

```
cv2.imread(filename, flags=None) -> retval
```

- ✓ filename: 불러올 영상 파일 이름 (문자열)
 - 상대 경로: 'cat.bmp', '../data/cat.bmp'
 - 절대 경로: 'c:\cat.bmp', '/home/id/cat.bmp'
- ✓ flags: 영상 파일 불러오기 옵션 플래그

cv2.IMREAD_COLOR	BGR 컬러 영상으로 읽기 (기본값) shape = (rows, cols, 3), dtype=uint8
cv2.IMREAD_GRAYSCALE	그레이스케일 영상으로 읽기 shape = (rows, cols), dtype=uint8
cv2.IMREAD_UNCHANGED	영상 파일 속성 그대로 읽기 (e.g.) 투명한 PNG 파일: shape = (rows, cols, 4)

- ✓ retval: 불러온 영상 데이터 (numpy.ndarray)

□ 영상 파일 저장하기

```
cv2.imwrite(filename, img, params=None) -> retval
```

- ✓ filename: 저장할 영상 파일 이름 (문자열)
- ✓ img: 저장할 영상 데이터 (numpy.ndarray)
- ✓ params: 파일 저장 옵션 지정 (속성 & 값의 정수 쌍)
 - e.g) JPG 파일 압축률을 90%로 지정하고 싶다면
[cv2.IMWRITE_JPEG_QUALITY, 90] 지정
- ✓ retval: 정상적으로 저장하면 True, 실패하면 False.

□ 새 창 띄우기 & 창 닫기

```
cv2.namedWindow(winname, flags=None) -> None
```

- ✓ winname: 창 고유 이름. 이 이름으로 창을 구분함.
- ✓ flags: 창 속성 지정 플래그

cv2.WINDOW_NORMAL	영상 크기가 창 크기에 맞게 지정됨
cv2.WINDOW_AUTOSIZE	창 크기가 영상 크기에 맞게 자동으로 변경됨 (기본값)

```
cv2.destroyWindow(winname) -> None  
cv2.destroyAllWindows() -> None
```

- ✓ winname: 닫고자 하는 창 이름
- ✓ 일반적인 경우 프로그램 종료 시 운영 체제에 의해 열려 있는 모든 창이 자동으로 닫힘

□ 창 위치 & 크기 지정

```
cv2.moveWindow(winname, x, y) -> None
```

- ✓ winname: 창 이름
- ✓ x,y: 이동할 위치 좌표

```
cv2.resizeWindow(winname, width, height) -> None
```

- ✓ winname: 창 이름
- ✓ width,height: 변경할 창 크기
- ✓ 참고 사항
 - 창 생성 시 cv2.WINDOW_NORMAL 속성으로 생성되어야 동작함.
 - 영상 출력 부분의 크기만을 고려함 (제목 표시줄, 창 경계는 고려되지 않음)

□ 영상 출력하기

```
cv2.imshow(winname, mat) -> None
```

- ✓ winname: 영상을 출력할 대상 창 이름
- ✓ mat: 출력할 영상 데이터 ([numpy.ndarray](#))
- ✓ 데이터 타입에 따른 출력 방식
 - uint8: 픽셀 값을 그대로 출력
 - uint16,int16: 픽셀 값을 255로 나눠서 출력
 - float32,float64: 픽셀 값에 255를 곱해서 출력
- ✓ 참고 사항
 - 만약 winname에 해당하는 창이 없으면 자동으로 cv2.WINDOW_AUTOSIZE 속성의 창을 새로 만들어서 영상을 출력함
 - Windows 운영체제에서는 Ctrl + C (복사), Ctrl + S (저장) 지원
 - 실제로는 cv2.waitKey() 함수를 호출해야 화면에 영상이 나타남.

□ 키보드 입력 대기

```
cv2.waitKey(delay=None) -> retval
```

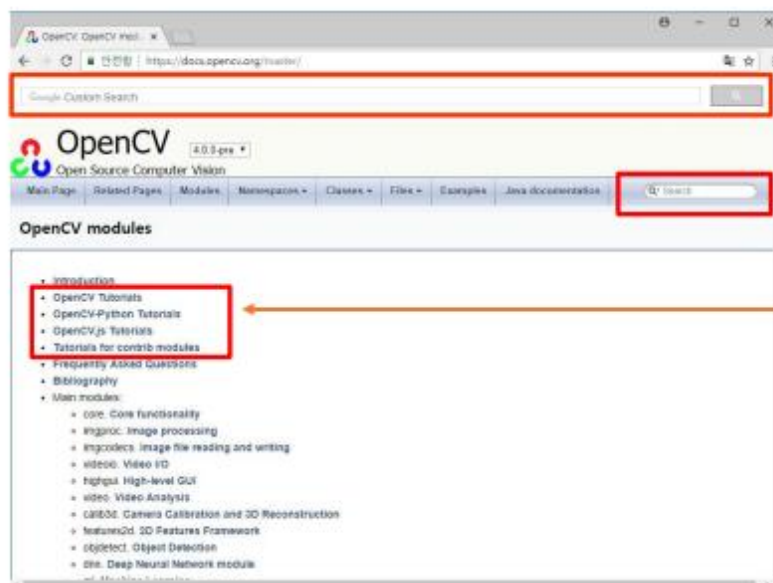
- ✓ delay: 밀리초 단위 대기 시간. delay ≤ 0 이면 무한히 기다림. 기본값은 0.
- ✓ retval: 눌린 키 값(ASCII code). 키가 눌리지 않으면 -1.
- ✓ 참고 사항
 - cv2.waitKey() 함수는 OpenCV 창이 하나라도 있을 때 동작함
 - 특정 키 입력을 확인하려면 ord() 함수를 이용

```
while True:
    if cv2.waitKey() == ord('q'):
        break
```

- 주요 특수키 코드: ESC 27, ENTER 13, TAB 9

□ OpenCV API 도움말 찾기

- ✓ OpenCV 최신 도움말: <http://docs.opencv.org/master/>
- ✓ OpenCV 도움말 웹페이지에서 우측 상단 검색창 활용



OpenCV 사이트
내부에서 구글 검색

OpenCV API 검색

Tutorials

Matplotlib을 이용한 영상 출력

□ Matplotlib 패키지

- ✓ 함수 그래프, 차트(chart), 히스토그램(histogram) 등의 다양한 그리기 기능을 제공하는 Python 패키지
- ✓ 보통 `matplotlib.pyplot` 모듈을 `plt` 이름으로 불러와서 사용
- ✓ Jupyter Notebook에서 사용할 때 유용

```
> pip install matplotlib
```

□ 컬러 영상 출력

- ✓ 컬러 영상의 색상 정보가 **RGB 순서**이어야 함
- ✓ `cv2.imread()` 함수로 불러온 영상의 색상 정보는 **BGR 순서**이므로 이를 RGB 순서로 변경해야 함 ⑦ `cv2.cvtColor()` 함수 이용

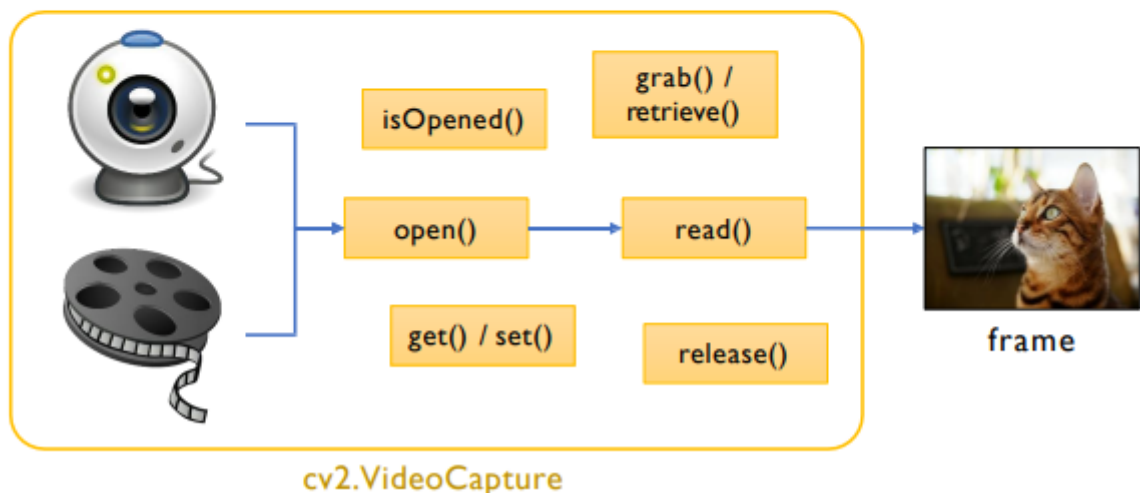
□ 그레이스케일 영상 출력

- ✓ `plt.imshow()` 함수에서 컬러맵을 `cmap='gray'` 으로 지정

카메라 & 동영상 처리

□ cv2.VideoCapture 클래스

- ✓ OpenCV에서는 카메라와 동영상으로부터 프레임(frame)을 받아오는 작업을 `cv2.VideoCapture` 클래스 하나로 처리함



OpenCV API

□ 동영상, 정지 영상 시퀀스, 비디오 스트림 열기

```
cv2.VideoCapture(filename, apiPreference=None) -> retval
```

- ✓ filename:
 - 비디오 파일 이름 (e.g. 'video.avi')
 - 정지 영상 시퀀스 (e.g. 'img_%02d.jpg')
 - 비디오 스트림 URL (e.g. 'protocol://host:port/script?params|auth')
- ✓ apiPreference: 선호하는 동영상 처리 방법을 지정
- ✓ retval: cv2.VideoCapture 객체

```
cv2.VideoCapture.open(filename, apiPreference=None) -> retval
```

- ✓ retval: 성공하면 True, 실패하면 False.

□ 카메라 열기

```
cv2.VideoCapture(index, apiPreference=None) -> retval
```

- ✓ index: camera_id + domain_offset(CAP_*) id
 - camera_id == 0 이면 시스템 기본 카메라
 - domain_offset == 0 이면 auto detect.
 - 기본 카메라를 기본 방법으로 열려면 index에 0을 전달.
- ✓ apiPreference: 선호하는 카메라 처리 방법을 지정
- ✓ retval: cv2.VideoCapture 객체

```
cv2.VideoCapture.open(index, apiPreference=None) -> retval
```

- ✓ retval: 성공하면 True, 실패하면 False.

□ 비디오 캡처가 준비되었는지 확인

```
cv2.VideoCapture.isOpened() -> retval
```

✓ retval: 성공하면 True, 실패하면 False.

□ 프레임 받아오기

```
cv2.VideoCapture.read(image=None) -> retval, image
```

✓ retval: 성공하면 True, 실패하면 False.

✓ image: 현재 프레임 (numpy.ndarray)

□ 카메라, 비디오 장치 속성 값 참조

```
cv2.VideoCapture.get(propId) -> retval
```

✓ propId: [속성 상수](#)

cv2.CAP_PROP_FRAME_WIDTH	프레임 가로 크기
cv2.CAP_PROP_FRAME_HEIGHT	프레임 세로 크기
cv2.CAP_PROP_FPS	초당 프레임 수
cv2.CAP_PROP_FRAME_COUNT	비디오 파일의 총 프레임 수
cv2.CAP_PROP_POS_MSEC	밀리초 단위로 현재 위치
cv2.CAP_PROP_POS_FRAMES	현재 프레임 번호
cv2.CAP_PROP_EXPOSURE	노출
cv2.CAP_PROP_ZOOM	줌(확대/축소 비율)

✓ retval: 성공하면 해당 속성 값, 실패하면 0.

□ 카메라, 비디오 장치 속성 값 참조

```
cv2.VideoCapture.set(propId, value) -> retval
```

- ✓ propId: 속성 상수
- ✓ value: 속성 값
- ✓ retval: 성공하면 True, 실패하면 False.

동영상 파일 저장하기

□ cv2.VideoWriter 클래스

- ✓ OpenCV에서는 cv2.VideoWriter 클래스를 이용하여 일련의 프레임을 동영상 파일로 저장할 수 있음
- ✓ 일련의 프레임은 모두 크기와 데이터 타입이 같아야 함

□ Fourcc (4-문자 코드, four character code)

- ✓ 동영상 파일의 코덱, 압축 방식, 색상 혹은 픽셀 포맷 등을 정의하는 정수 값
- ✓ <http://www.fourcc.org/codecs.php>

cv2.VideoWriter_fourcc(*'DIVX')	DIVX MPEG-4 코덱
cv2.VideoWriter_fourcc(*'XVID')	XVID MPEG-4 코덱
cv2.VideoWriter_fourcc(*'FMP4')	FFMPEG MPEG-4 코덱
cv2.VideoWriter_fourcc(*'X264')	H.264/AVC 코덱
cv2.VideoWriter_fourcc(*'MJPG')	Motion-JPEG 코덱

OpenCV API

□ 저장을 위한 동영상 파일 열기

```
cv2.VideoWriter(filename, fourcc, fps, frameSize, isColor=None)
-> retval
```

- ✓ filename: 비디오 파일 이름 (e.g. 'video.mp4')
- ✓ fourcc: fourcc (e.g. cv2.VideoWriter_fourcc(*'DIVX'))
- ✓ fps: 초당 프레임 수 (e.g. 30)
- ✓ frameSize: 프레임 크기 (e.g. [640, 480])
- ✓ isColor: 컬러 영상이면 True, 그렇지 않으면 False.
- ✓ retval: cv2.VideoWriter 객체

```
cv2.VideoWriter.open(filename, fourcc, fps, frameSize,
                     isColor=None) -> retval
```

- ✓ retval: 성공하면 True, 실패하면 False.

□ 비디오 파일이 준비되었는지 확인

```
cv2.VideoWriter.isOpened() -> retval
```

- ✓ retval: 성공하면 True, 실패하면 False.

□ 프레임 받아오기

```
cv2.VideoWriter.write(image) -> None
```

- ✓ image: 저장할 프레임 (numpy.ndarray)